

Apache Camel for Enterprise Integration

Focus: Use Apache Camel to process files and connect systems.

1. What is Apache Camel?

Apache Camel is an open-source integration framework that helps connect different systems, applications, and data formats using a consistent set of Enterprise Integration Patterns (EIPs).

- Acts as “glue code” between systems (files, databases, REST APIs, message queues, etc.)
- Uses a simple, readable DSL (Domain Specific Language) available in Java, XML, and YAML
- Core idea: define routes that say “take data from here, do something with it, send it there”
- Runs standalone, embedded in Spring Boot, or on Quarkus/Karaf

Why it's used in enterprises

- Avoids writing repetitive boilerplate connector code
- Standardizes how systems talk to each other
- Huge library of pre-built connectors (300+) for databases, queues, APIs, cloud services

2. Camel Route

A route is the core building block in Camel — it defines the path a message takes from a source to a destination.

Syntax explained

```
from("file:input")
  .to("file:output");
```

- **from(...)** — **declares** the source endpoint; this is where Camel listens for incoming messages (a “consumer”)
- **.to(...)** — **declares** the destination endpoint; this is where the message is sent (a “producer”)
- The **.** chaining means each step happens in sequence on the same message
- A route is defined inside a RouteBuilder class by overriding the configure() method

Step-by-step demo: Your first route

```
import org.apache.camel.builder.RouteBuilder;

public class MyFirstRoute extends RouteBuilder {
    @Override
    public void configure() throws Exception {
        from("timer:helloTimer?period=5000") // Step 1: trigger every 5 seconds
            .setBody(constant("Hello from Camel!")) // Step 2: set message content
            .to("log:myLogger"); // Step 3: print to console log
    }
}
```

What happens:

1. timer:helloTimer?period=5000 fires an event every 5000 ms (5 seconds)
2. setBody(constant(...)) sets a fixed text as the message body
3. to("log:myLogger") prints the message to the console

Run it (Spring Boot example):

1. Add camel-spring-boot-starter and camel-timer-starter to pom.xml
2. Place the route class anywhere in your Spring Boot project (Camel auto-detects RouteBuilder beans)
3. Run mvn spring-boot:run
4. Console prints "Hello from Camel!" every 5 seconds

3. Camel Endpoint

An endpoint represents a connection point to a specific system or resource, written as a URI-style string.

Syntax explained

```
component:context-path?option1=value1&option2=value2
```

- **component** — the type of system (e.g., file, kafka, jms, http)
- **context-path** — specific target (folder name, topic name, queue name, URL path)
- **?options** — query-string style settings, separated by &

Examples

```
file:data/inbox?noop=true
kafka:my-topic?brokers=localhost:9092
rest:get:/api/orders
jms:queue:orders
timer:myTimer?period=1000
```

Step-by-step demo: Comparing consumer vs producer endpoint

```
from("file:data/inbox")    // endpoint as CONSUMER (reads files)
    .to("file:data/outbox"); // endpoint as PRODUCER (writes files)
```

1. Camel polls data/inbox folder for new files
2. When a file appears, it reads it into a message
3. The message is sent to data/outbox, which writes the file there

4. Camel Processor

A Processor is a Java interface that lets you write custom logic to inspect or modify a message inside a route, for cases the built-in DSL can't handle directly.

Syntax explained

```
.process(exchange -> {
    String body = exchange.getIn().getBody(String.class);
    exchange.getIn().setBody(body.toUpperCase());
})
```

- **exchange** — the Camel object wrapping the message plus metadata (headers, properties)
- **exchange.getIn()** — gets the “in message” (the incoming request)
- **.getBody(String.class)** — extracts the body and converts it to a String
- **.setBody(...)** — replaces the message body with new content
- The lambda exchange -> { ... } is shorthand for implementing the Processor interface

Step-by-step demo: Uppercase converter

```
from("file:data/inbox?noop=true")
    .process(exchange -> {
        String body = exchange.getIn().getBody(String.class);    // Step 1: read
original text
        exchange.getIn().setBody(body.toUpperCase());            // Step 2: transform
it
    })
    .to("file:data/outbox");                                     // Step 3: save
result
```

What happens:

1. Camel reads a text file from data/inbox
2. The processor converts the file content to uppercase

3. The transformed content is written to data/outbox

Try it: Drop a file hello.txt containing "hello world" into data/inbox → check data/outbox for "HELLO WORLD".

5. File Processing

Camel's file component automates reading, writing, moving, and filtering files — common in batch and ETL-style integration.

Syntax explained

```
from("file:data/inbox?noop=true&include=*.csv&move=processed")
```

- **noop=true** — “no operation”: don't delete or move the file after reading (good for testing)
- **include="*.csv"** — only process files matching this pattern
- **move=processed** — after successful processing, move the file into a processed subfolder
- **\${header.CamelFileName}** — built-in Camel header holding the current file's name

Step-by-step demo: Watch a folder and archive processed files

```
from("file:data/inbox?include=*.txt&move=processed")
  .log("Processing file: ${header.CamelFileName}")
  .to("file:data/outbox");
```

1. Camel watches data/inbox for .txt files
2. When found, it logs the file name using the Simple language expression `${header.CamelFileName}`
3. Copies the file content to data/outbox
4. Moves the original file into data/inbox/processed (so it's not reprocessed)

Step-by-step to run:

1. Create folder data/inbox
2. Start the Camel application
3. Drop report.txt into data/inbox
4. Observe console log: "Processing file: report.txt"
5. Check data/outbox for the copy, and data/inbox/processed for the archived original

6. Basic Data Transformation

Camel can convert between formats (JSON, XML, CSV) using marshal/unmarshal, and manipulate fields using the Simple expression language.

Syntax explained

```
.unmarshal().json()
.setBody(simple("${body[name]}"))
.marshal().json()
```

- **.unmarshal()** — converts raw data (e.g., JSON text) into a Java object (like a Map or POJO)
- **.marshal()** — converts a Java object back into raw data (e.g., JSON text)
- **simple("\${body[name]}")** — Simple language expression to pull the name field from the body map
- Requires a data format dependency, e.g. camel-jackson for JSON

Step-by-step demo: Extract a field from JSON and reformat

Input file data/inbox/order.json:

```
{ "id": 101, "customer": "Asha", "amount": 250 }
```

Route:

```
from("file:data/inbox?include=*.json")
    .unmarshal().json()                                // Step 1: JSON text -> Java
Map
    .process(exchange -> {
        Map<String, Object> body = exchange.getIn().getBody(Map.class);
        body.put("status", "RECEIVED");                // Step 2: add a new field
    })
    .marshal().json()                                  // Step 3: Java Map -> JSON
text
    .to("file:data/outbox");
```

1. Camel reads order.json and converts it into a Map<String, Object>
2. The processor adds a new key status: RECEIVED
3. The map is marshalled back into JSON text
4. Output file in data/outbox looks like the block below

```
{"id":101,"customer":"Asha","amount":250,"status":"RECEIVED"}
```

7. REST to Kafka Integration

A common enterprise pattern: expose a REST endpoint that accepts incoming data and publishes it to a Kafka topic for downstream consumers.

Syntax explained

```
rest("/api/orders")
  .post("/")
  .to("direct:processOrder");
```

- **rest("/api/orders")** — defines a base REST path
- **.post("/")** — accepts HTTP POST requests on that path
- **.to("direct:processOrder")** — forwards the request internally to another route named processOrder
- **direct:** — an in-memory, synchronous link between two routes inside the same Camel context (not exposed externally)
- **kafka:orders-topic?brokers=localhost:9092** — sends the message to a Kafka topic on the given broker

Step-by-step demo: Order API → Kafka topic

```
// Route 1: Expose REST endpoint
rest("/api/orders")
  .post("/")
  .to("direct:processOrder");

// Route 2: Process and publish to Kafka
from("direct:processOrder")
  .log("Received order: ${body}")
  .marshal().json()
  .to("kafka:orders-topic?brokers=localhost:9092");
```

Steps to test:

1. Start the Camel application (REST component such as camel-platform-http or camel-netty-http must be on the classpath)
2. Start a local Kafka broker on localhost:9092
3. Send a test request using the curl command below

```
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -d '{"id":101,"customer":"Asha","amount":250}'
```

1. Camel logs: "Received order: {id=101, customer=Asha, amount=250}"
2. Message is published to the orders-topic Kafka topic

3. Verify using a Kafka consumer with the command below

```
kafka-console-consumer.sh --topic orders-topic --bootstrap-server localhost:9092 --from-beginning
```

Quick Recap

Concept	Purpose	Key Syntax
Route	Defines flow from source to destination	<code>from(...).to(...)</code>
Endpoint	URI representing a system/resource	<code>component:path?options</code>
Processor	Custom Java logic on a message	<code>.process(exchange -> {...})</code>
File Component	Read/write/move files automatically	<code>file:path?include=&move=</code>
Marshal/Unmarshal	Convert between data formats	<code>.marshal().json() / .unmarshal().json()</code>
REST + Kafka	Expose API, stream data into Kafka	<code>rest(...).post(...) -> to("kafka:...")</code>

Suggested Flow (Demo Order)

1. Run the timer → log route to show the basic route concept live
2. Run the file processing demo (drop a file, watch it move to processed)
3. Add a processor to transform file content (uppercase demo)
4. Show JSON transformation (add a field to an order JSON)
5. Finish with the REST → Kafka demo as the capstone, using curl + a Kafka consumer to prove end-to-end flow